

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

September 22, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	September 22, 2026	45 pages

Tiivistelmä — Referat — Abstract

Context. Radio access network (RAN) control-plane software is written mostly in C++ because C++ is fast. The cost is memory bugs, and bugs when many threads share data. Rust is meant to be about as fast, and to reject those bugs when the program is compiled. It is not yet shown that this holds when a late call matters as much as a fast average.

Objective. The thesis compares modern C++ and Rust on a user equipment (UE) Context Management service. The service stores the state of each connected device, as a 5G base-station control plane (gNB-CU-CP) does. A comparison of average speed leaves four questions open. How late are the slowest calls? Which C++ bug types does Rust refuse to compile? What do Rust's safety checks cost? What does it cost to add Rust one piece at a time?

Methods. One specification is written first, before either language is chosen, and then built in C++ and in Rust. The algorithms, the thread model, the memory allocator, and the build settings match. Where the languages cannot be written the same way, the difference is logged. A made-up workload is raised until the service is overloaded. Both versions pass the same tests. The study also injects known bugs, and builds a foreign function interface (FFI) so C++ can call Rust.

Results. (1) Safe Rust refuses to compile all seven injected memory-safety and data-race bug types. The same seven compile in C++. They show up only when the program runs, and only if a checking tool catches them. (2) Seven further bug types fail in both languages. C++ tools report two of them that Rust's tools do not. (3) C++ can call the Rust service through a thin C interface. All of the `unsafe` Rust sits in that interface, and the extra time is at most 10.91% of a real signalling call. (4) A full request spends only 0.250% of its time on work the language does, so that clock cannot say which language is faster. Timed on its own, that core work is 1.6× faster in Rust. (5) At 56 kilo-operations per second (kops/s) a typical call is 1.3 μs faster in Rust. That gap is too small to blame on the language. Among the slowest calls, the slowest Rust build is still 50 μs faster than the fastest C++ build. (6) Once the service is overloaded, one clock shows Rust ahead at 448 kops/s and C++ ahead by 1.7× at 784. Moving a recording step out of the timed window removes both gaps. The recorder is the likely cause, so how late the service itself is under overload stays unknown.

Conclusions. While the service can still keep up with the offered load, the useful choice is which bug types a team wants the compiler to forbid. Speed does not decide it. Rust removes data races. It does not remove every bug that comes from threads. The cost measured here is the crossing from C++ into Rust. Design time and training time were not measured. A test at normal load misses the overload boundary. A recorder inside the timed loop can create a false gap once that boundary is passed.

Avainsanat — Nyckelord — Keywords

Rust; C++; RAN control plane; tail latency; memory safety; fault injection, incremental adoption

ACM Computing Classification System (CCS)

Software and its engineering → Software notations and tools → General programming languages

Computer systems organization → Real-time systems → Real-time system specification

Networks → Network types → Mobile networks

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures iii

List of Tables iv

List of acronyms v

1 Introduction 1

1.1 Motivation . . . 2
1.2 Industrial context . . . 3
1.3 Problem statement . . . 4
1.4 Research questions and hypotheses . . . 5

2 Background: Systems Languages for Latency-Critical Software 7

2.1 RAN control-plane systems and the gNB-CU-CP . . . 7
2.2 Research gap . . . 8

3 Research Methodology 11

3.1 The UE Context Management subsystem . . . 11
3.2 Research approach . . . 12
3.3 Methods per research question . . . 14
3.4 Fairness as a controlled variable . . . 15
3.5 Statistical treatment . . . 15
3.6 Reproducibility and research artefacts . . . 16

4 Design and Implementation 17

5 Experimental Setup and Evaluation 19

5.1 Testbed setup . . . 19
5.2 Workload and measurements . . . 20
5.3 Results . . . 21
5.3.1 Below saturation (RQ1, RQ3) . . . 21
5.3.2 Past saturation (RQ1) . . . 23

5.3.3 Resource use (RQ3) . . . 24
5.3.4 Safety via fault injection (RQ2) . . . 25
5.3.5 Migration (RQ4) . . . 27

6 Discussion **28**
6.1 Summary of findings . . . 28
6.2 Discussion on the research questions . . . 28
6.3 Revisiting the hypotheses . . . 31
6.4 Engineering guidelines for incremental adoption . . . 31
6.5 Where this study sits . . . 32
6.6 Ethical and professional reflection . . . 32

7 Conclusion and Future Work **34**

Statement on the Use of AI Tools **37**

Bibliography **38**

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 8
2.2 What the neighbouring studies control, and what they leave varying 10

3.1 UE-context lifecycle state machine 11

4.1 Shared-nothing, thread-per-shard architecture 17

5.1 The two workloads, and the lifecycle the mixed one walks 20
5.2 The load sweep: tail latency and achieved throughput 21
5.3 Where the end-to-end clock goes 22
5.4 The end-to-end operation decomposed, against what the sweep can resolve 23
5.5 Campaign-clock load regimes on the offered-load axis 23
5.6 Campaign and deferred-fill throughput on two captured setups 25
5.7 Resource use, and the two effects inside peak RSS 26
5.8 Where the guarantee stops: the outcome matrix 27
5.9 The C++-to-Rust FFI migration seam 27

6.1 Where Rust's compile-time guarantee stops 30

List of Tables

3.1 How each research question is answered 14

5.1 Per-operation latency at the representative load 22

List of acronyms

Each acronym below is also written out where the text first uses it.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
ASN.1	Abstract Syntax Notation One
CPU	Central processing unit
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DPDK	Data Plane Development Kit
EU	European Union
FFI	Foreign function interface
FIFO	First in, first out
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
GCC	GNU Compiler Collection
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane, the domain this work sits in
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
KPI	Key performance indicator
kops/s	Kilo-operations per second
KVM	Kernel-based Virtual Machine
MPSC	Multi-producer, single-consumer
NG-RAN	Next-generation radio access network
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here

1 Introduction

Mobile networks are moving from special hardware to software on ordinary servers. Jobs that once ran on equipment built for one purpose are now programs on general-purpose machines. The radio access network (RAN) is the part that talks to phones over the air. For a long time it was the most hardware-heavy part of the network. It is part of this shift too.

In 5G, the base station is called a next-generation NodeB (gNB). It is split into a central unit and one or more distributed units. The central unit is split again, into a control plane and a user plane [1, 2]. The control plane decides what should happen. The user plane carries the user's data. Open-RAN takes the same split further. The base station becomes software from several vendors, joined by open interfaces, and it can run on ordinary machines [3]. The control plane in this split is the gNB central unit control plane (gNB-CU-CP). It is a large program. It remembers state, and many threads run in it at once. It tracks every connected device. The standards call each device a user equipment (UE). It runs Radio Resource Control (RRC) signalling, which is the conversation that sets up and changes a connection [4]. It also handles mobility and handovers, which move a phone from one cell to another. All of this has to finish inside a strict time limit.

One site can run thousands of these procedures per second. Each one has to finish within a few milliseconds. That is the budget the 5G system sets for control-plane signalling [5]. A live control plane may track millions of active devices at once. As this software moves into the cloud and is split across vendors, its correctness, reliability, and timing decide the quality of the whole network. An outage here affects every device the site serves. So does a tail-latency spike. Tail latency means a few calls take much longer than the rest, even when the average still looks fine [6, 7].

This control-plane software is written mostly in C++ today. The reason is practical. C++ gives the programmer close control of the machine, mature tools, and the speed that a late call cannot afford to lose. That speed has a known cost. In C++, the programmer must manage memory and threads by hand. Decades of experience show that this often goes wrong. The result is bugs such as use-after-free, where the program uses memory after it has been freed, a double-free, where the same memory is freed twice, and a data race, where two threads use the same data without a rule for who goes first. These bugs are expensive to find, dangerous in a live network, and a large share of security flaws in industry practice [8, 9]. Rust is the first mainstream systems language that tries to leave

this trade-off behind. It checks memory safety and data-race freedom when the program is compiled, on a machine-checked foundation [10, 11]. It is also meant to be about as fast as C++ [12]. It is not well understood whether that promise holds for a telecom control plane. There, a predictable delay under overload matters as much as raw speed. This thesis, carried out at Nokia, asks that question directly. It builds the same RAN subsystem twice, once in modern C++ and once in Rust. The subsystem is a UE Context Management service. It stores the state of each device, as a gNB-CU-CP does. The two versions are then compared under a made-up signalling workload that follows the shape of real procedures.

Three terms come back throughout. The *gNB-CU-CP* is the kind of control-plane software this work is about. *UE Context Management (UECM)* is the small service that was rebuilt and measured. *Tail latency* is the slow end of the delay, not the average. It is reported as percentiles: p99 means 99% of calls were at least this fast, and p99.9 and p99.99 look further into the slowest calls. That slow end, more than average speed, decides whether the system meets its time budget [6, 7].

1.1 Motivation

Three trends make this comparison timely. The first is that the RAN is becoming cloud software. Control-plane functions now run on ordinary servers. The software lives longer, more threads run at once, and it must still meet its time budgets without hardware that was built to be predictable. Open-RAN splits the base station into software from several vendors [3]. A slowdown, or a bug that corrupts memory, can then affect the whole network. The choice of programming language is an engineering decision.

The second trend is artificial intelligence (AI). AI used to be only a tool. It helped choose radio settings, such as how much power to send. That tool did not have to run on the radio's own server. Now AI programs also run on that server, next to the radio software [13]. Both run at the same time, so they share one processor. International Mobile Telecommunications-2030 (IMT-2030) is the 6G framework of the International Telecommunication Union (ITU). It lists AI together with communication as one named job for 6G [14]. The architecture forming around that job is AI-RAN. In it, AI work and radio work share one accelerated server [15]. A control plane there no longer has the machine to itself. It shares the central processing unit (CPU), memory, and cache with inference work that it does not schedule. When that neighbour takes capacity, the control plane saturates sooner. It stops keeping up with demand at a lower load than it would on its

own. Chapter 5 finds the widest gaps on the campaign clock past that point. Later diagnostic sweeps do not reproduce either gap, so they cannot be read as a ranking of the two services.

The third trend is the move toward languages that are safe with memory. Large software organisations report that most of their serious security vulnerabilities come from memory-safety errors, and that adopting memory-safe languages reduces them [8, 9, 16, 17]. Independent work argues that memory safety now needs standard requirements if buyers are to demand it at all [18]. Cybersecurity agencies urge a move away from C and C++. The European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [19, 20, 21]. The Act names no programming language. Direct evidence for a RAN control plane is still missing. Providing that evidence is the motivation for this work.

1.2 Industrial context

This thesis is carried out at Nokia. Nokia is one of the small number of vendors that build, maintain, and evolve production RAN software from end to end. The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025. Concentration, measured by the Herfindahl–Hirschman Index, is at a ten-year high [22, 23]. The subsystem this thesis studies, UECM, is a small model of the per-device state work a gNB-CU-CP does.

In the architecture of the 3rd Generation Partnership Project (3GPP), the gNB-CU-CP is the control-plane half of the central unit. It hosts RRC and manages the life of each UE context [1, 4], under the millisecond budgets the 5G system sets for control signalling [5]. Chapter 2 places that control plane inside the split 5G base station, the gNB. Because it is written in C++, it inherits that language's speed and its memory and thread hazards. The three trends of Section 1.1 meet here. Software of this class runs on live operator networks. The language choice is a production concern.

UECM is small enough to rebuild faithfully and to reason about in full. It still has the properties that make control-plane code hard. The implementation follows a public, frozen specification. It does not follow Nokia's private code, so readers outside Nokia can repeat the comparison.

A decision with a generational horizon. A language choice in software of this class outlives the product generation it was made for. 3GPP has assigned Release 20 to the

6G studies and Release 21 to the normative work, the work that becomes a binding specification. The ITU asks for technology proposals in early 2029 and for specification submissions by mid-2030 [24]. That work covers moving from 5G and working alongside it. A control plane written now will be maintained next to the next generation. It will not be retired when 6G arrives. IMT-2030 still names security and resilience as a design principle, and it still carries the ultra-reliable low-latency scenario, in a stronger form [14].

The measurements here use a current-generation subsystem and requirements that have already been published. They make no claim about requirements that have not been written yet.

A concrete failure mode. A handover moves a connected UE from one cell to another, so the session can continue. When many UEs do that at once, the control plane is under a mobility storm. Each UE has a stored record, the UE context. One of those UEs is released, so its record is deleted. At the same time, a message for that UE is still being handled. The next read of the record is a use-after-free. In C++ this still compiles. Tests that do not hit that overlap can pass. When many calls run at once, the bug can corrupt another UE's state, crash the thread that is serving the call, or make a few calls much slower than the rest. That jump in the slowest calls is a tail-latency spike. In safe Rust the same design does not compile. Ownership and borrowing reject the dangling reference, the pointer left behind after the record was deleted. That is why this thesis measures both *safety*, meaning which bugs are prevented and when, and *predictability*, meaning tail latency under overload. Average throughput alone would miss both.

1.3 Problem statement

Two facts about this domain make the language choice matter. First, C++ leaves memory and threads to the programmer. That exposes the software to bugs such as use-after-free, double-free, data races, and iterator invalidation, which is using a pointer into a collection after the collection has changed. These bugs compile without a warning. At the scale of a telecom product they are costly to find and fix. Second, RAN time budgets are met or missed by the slowest calls under load and overload, not by average speed [6]. That slow end is easy to make worse and hard to measure correctly. Rust claims to remove the first kind of bug at compile time, and to stay competitive on the second. That claim rests mostly on general benchmarks, not on telecom software that keeps state, uses many threads, and must stay predictable when it is overloaded.

The concrete problem is this. Vendors are under growing pressure, technical and now also legal, to adopt languages that are safe with memory. For the RAN control plane they still lack direct evidence from a controlled comparison. Three questions are open. Can a Rust version of a stateful, multi-thread control-plane service match modern C++ on tail latency under overload, which is a different question from average speed? Do Rust's compile-time checks actually remove the bug types that matter in the existing C++ code? What does it cost to add Rust one piece at a time inside a large C++ codebase? The evidence that exists is mostly general-purpose and about the average case [12, 25]. Answering those three questions under controlled, repeatable conditions is the problem this thesis takes on.

1.4 Research questions and hypotheses

The thesis asks four questions. The first is about how predictable the delay is. The second is about which bugs disappear. The third is about the cost in speed and memory. The fourth is about adding Rust one piece at a time.

RQ1 As the load rises, and then goes past the point where the service can keep up, how do Rust and modern C++ compare on delay? The comparison uses the median, the tail at p99, p99.9 and p99.99, and the slowest call.

RQ2 Which memory-safety and concurrency bugs that compile in C++ does the Rust version reject at compile time? For the bugs it does not reject, when are they found, and what does finding them cost?

RQ3 What, if anything, do Rust's safety checks cost? The checks include bounds checks, extra copies forced by ownership, and the cost of abstractions. The measures are throughput, CPU use, and memory use.

RQ4 What does it take to add Rust one piece at a time inside an existing C++ RAN codebase? The question covers effort, calling Rust from C++ through a foreign function interface (FFI), and the risks of that boundary.

Three hypotheses were written down before any measurement.

H1 For this workload the two languages are equally fast while the service can still keep up. Any difference shows up only among the very slowest calls.

H2 Safe Rust stops, at compile time, the memory-safety and concurrency bugs that C++ can only try to catch after the program has been built.

H3 Calling the Rust service from C++ through the FFI adds only a small share of one signalling call's time. The real cost of adding Rust a piece at a time is engineering work, not slower calls.

Chapter 6 checks all three hypotheses against the measurements.

The RQ2 list has fourteen bug types. Safe Rust is expected to stop seven of them at compile time. It is expected not to stop the other seven. That split is the boundary the study is looking for. It is more than a check that the guarantee works. The seven types on the far side of the boundary are these. A deadlock from taking locks in the wrong order. A lost update, where two writers each read a value and both write back, so one change disappears. A leak from a cycle of references. An overflow that a release build defines as wrapping around. The blast radius of one failed worker, meaning how much else breaks when that worker dies. A double-free at the boundary between the two languages. A forgotten lock guard, meaning the code takes a lock and never releases it.

An earlier study estimated that boundary from a catalogue, not from programs that were built and run. The catalogue is the Common Weakness Enumeration (CWE), a long public list of bug types. The study marked each type as one that safe Rust would stop, or would not stop. It then counted real product bugs in 68 open-source C projects. Of the bugs that could be classified, 58.2% sat in types that safe Rust would have made almost impossible to write [26]. That share is a best case. It assumes the new code stays in safe Rust, with no `unsafe` blocks. A type counts as stopped if the mapping says so, even if no compiler has rejected a program. RQ2 asks the same question of programs that are built and run. That is how it can say when the types that stay open are caught, and what it costs to catch them.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The next-generation radio access network (NG-RAN) is the setting for this comparison. Phones move, and signalling can spike, so the event rate is high. The delay limits are tight, and they have to stay stable. The 5G service and RAN specifications state those limits as key performance indicators (KPIs) [5, 27]. The software lives for a long time and keeps growing. Many threads run at once. The limits are on the delay of one signalling call. A late call still misses them when the average call is on time. So the worst case, and the slow tail of the delay, matter more than the average. That is why this thesis measures the tail.

The gNB is split in two ways. The first split is central versus distributed. The gNB runs a protocol stack: a set of layered radio protocols, from the air interface up to RRC signalling. A central unit (gNB-CU) hosts the higher layers. One or more distributed units (gNB-DU) host the lower layers. Those lower layers sit close to the radio and are tight on time. The second split is inside the central unit, between control and user data. It separates the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts talk over standard links. NG goes toward the 5G core. F1 sits between the central unit and the distributed unit. E1 sits between the control plane and the user plane [1, 2, 28]. Open-RAN pushes the same links further. Each part can then come from a different vendor and run as software on ordinary machines [3]. Figure 2.1 shows the part studied here, UE Context Management (UECM), inside the gNB-CU-CP. UECM owns the control-plane state of each UE. It also handles RRC signalling and mobility. These are core control-plane jobs, so UECM is a fair example of a stateful subsystem. It does not stand for the whole control plane.

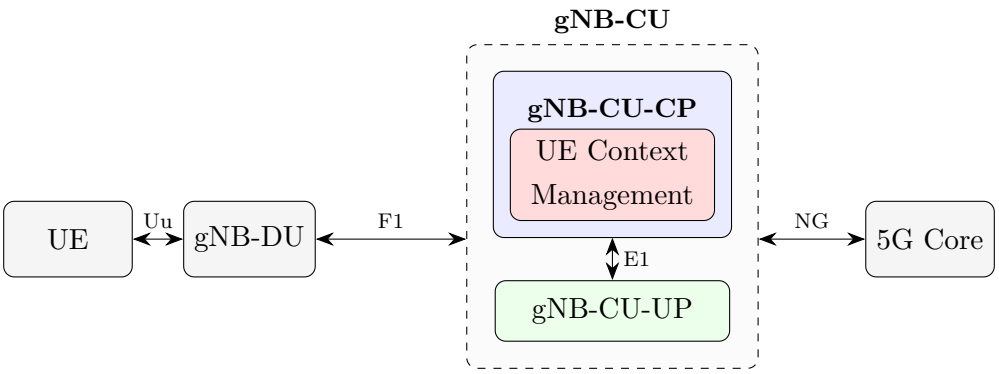


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. UE Context Management is highlighted inside the gNB-CU-CP.

2.2 Research gap

The RAN control plane has many threads and tight delay limits. What sets the quality of the network is the slow tail under too much load, not the average throughput. C++ gives the control and the speed this domain needs. It also leaves whole families of memory bugs and thread bugs to be found after the program has been built. Rust promises to block those families when the code is compiled, with no extra cost at run time. It also offers a way to adopt it step by step, by calling Rust through the C application binary interface (ABI). An ABI is the binary contract that lets one compiled program call another.

The domain is not unstudied. The amount of earlier work is easy to miss, because almost none of it is in the usual paper indexes. Five master's theses, all done at Ericsson and hosted by the vendor, have compared languages for RAN and telecom software. One weighs run-time cost against how fast people can build, across C++, Rust, and Go [29]. One asks whether Rust could replace C in RAN node management [30]. One builds a packet processor in async Rust, with one thread per core, and measures it against the Data Plane Development Kit (DPDK) program it copies [31]. DPDK is a library for moving packets very quickly. One measures median and p99 delay for control-plane steps across five async frameworks [32]. The last builds matched C++ and Rust versions of an Abstract Syntax Notation One (ASN.1) decompiler and compares them with statistical tests [33]. ASN.1 is the notation used to describe the messages these protocols exchange. All five are directly relevant. They do not all favour Rust. Olsson and Stenström's packet processor reaches a fifth to a half of DPDK's throughput, with much worse latency [31]. Arvidsson and Kaller's Rust p99 rises more steeply once the rate passes a threshold [32]. Karlbäck and Orö pick C++ where speed is critical. They still find Rust lightest on peak memory [29]. Olsson's management software is as fast as C, or faster, and it uses more

peak memory [30]. Emberg and Wadsten's pair is comparable in one session and 85–91% worse in the next [33].

Three of those studies change the language together with the run-time and the thread model. A measured gap then cannot be blamed on the language alone. It may come from the framework [29, 31, 32]. In the packet-processor study the DPDK program does two to five times as much work [31]. A compiler does not move throughput that far. The run-time does. Olsson's management study and Emberg and Wadsten do not change the run-time when they change the language [30, 33]. They report run time and peak memory, not the slow tail under rising load. Emberg and Wadsten come closest in method to this thesis. They still measure a stateless transform, a conversion that does not remember anything between calls. It is not a stateful part with many threads. Figure 2.2 places the five vendor theses beside the general-purpose comparisons [34] and the matched systems-software pairs [35, 36]. Each group holds some of what a fair test would need. None holds all of it.

That concentration is itself part of the gap. All five studies come from one vendor. Their conclusions have never been tested against another firm's software, targets, or house rules. So what is known about this question is what one firm's frameworks say about it. The published record on the other side is thinner. There is no study of Rust among the indexed papers of the vendor hosting this work. The nearest peer-reviewed stance from that side argues for making memory-error checking practical at run time. It does not argue for replacing that checking with a guarantee at compile time [37]. That run-time strategy is the one this thesis tests against.

What is missing is a direct, controlled test of a typical control-plane part. The open questions are these. Does Rust hold its tail delay under too much load against modern C++, when the thread model, the operations, and the data layout stay fixed? Which bug types does it remove, and at what stage are the rest caught? What does its safety model cost in throughput, CPU, and memory? What does step-by-step adoption cost in practice? Closing this gap is the purpose of the four research questions. The next chapter grounds them in the part under study, the bug types, and the timing at stake. It also sets out the method.

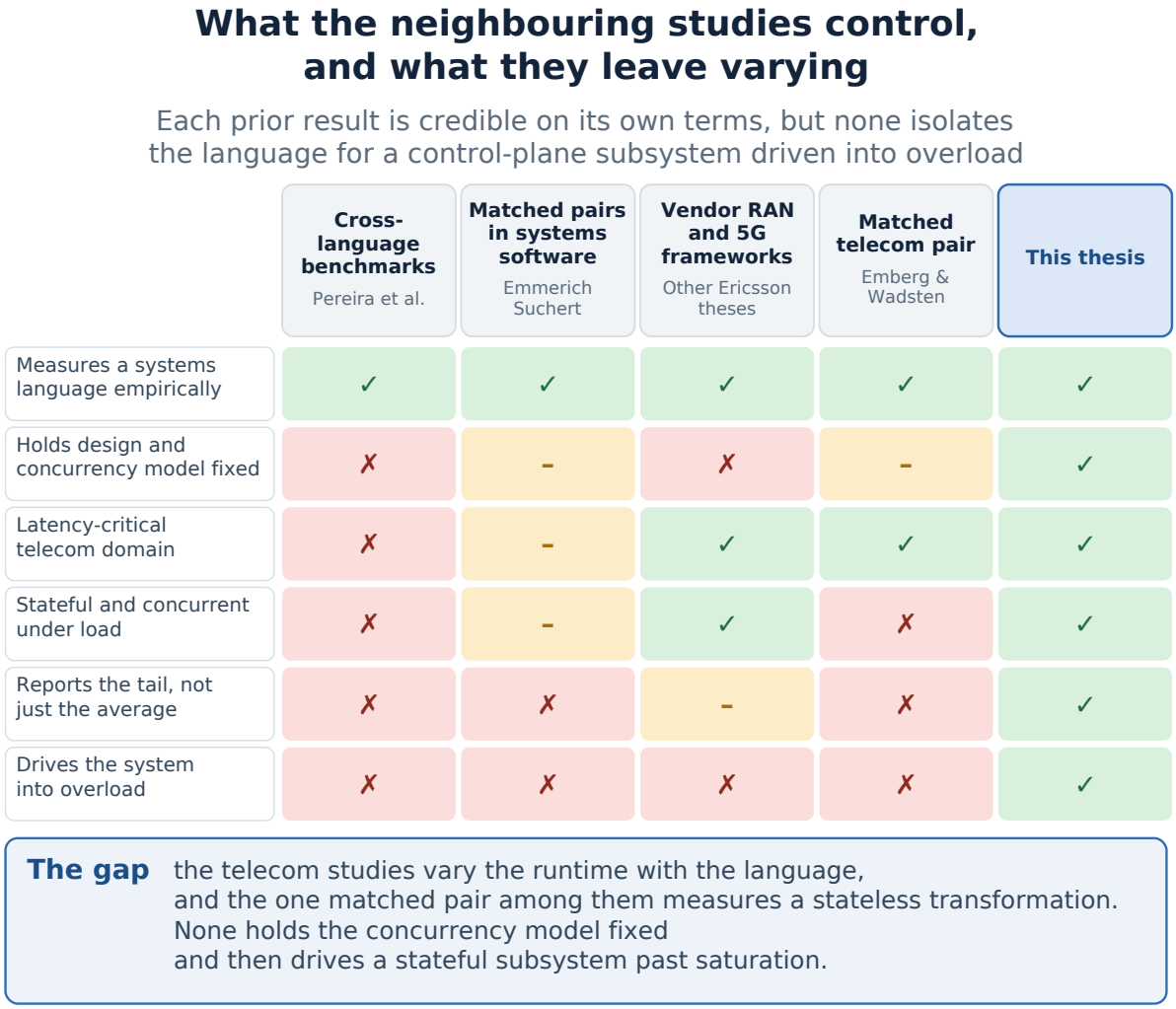


Figure 2.2: What the neighbouring studies control, and what they leave varying. Each row is a property a controlled answer would need, and each column is a group of studies. The vendor-RAN column is the Ericsson theses other than Emberg and Wadsten, who have their own column because they hold the design closer.

3 Research Methodology

The last chapter set out the gap. This chapter says how the study closes it.

3.1 The UE Context Management subsystem

The part under study is a UE Context Management service. Its operations follow the life of a UE context, the execution of RRC procedures [4], and the mobility and handover work that the 3GPP design assigns to the control-plane part of a split gNB (Section 2.1). Figure 3.1 shows that life cycle. Both versions encode it the same way. The mixed workload of Section 5.2 walks each UE through that machine.

A context starts Idle. A setup moves it to Connected. From Connected it can be reconfigured and then return, or it can be released. A release from Idle is also allowed. Signalling and adding a data radio bearer stay in Connected. An operation that is illegal in the current state returns an error and does not change the state.

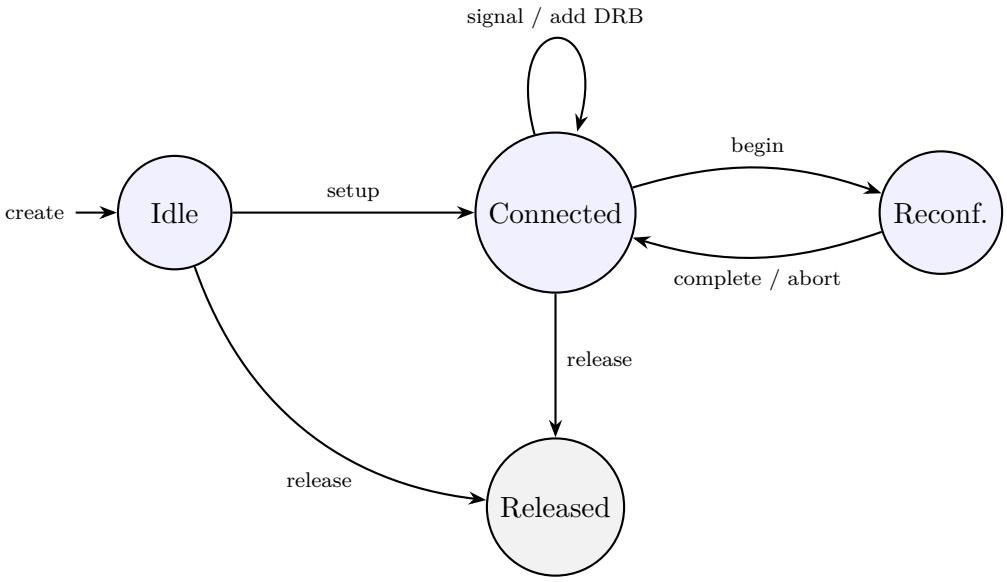


Figure 3.1: UE-context lifecycle state machine. Operations invalid for the current state return an error instead of changing state.

UECM is core RAN layer-3 work. Layer 3 is the signalling layer above the radio itself. It has the traits that make live control-plane code hard. Each UE has state that changes. The order of operations is strict. Many UEs run at once. The delay targets are the ones the 5G

system sets for control-plane procedures [5, 27]. Findings about how a language handles those traits apply most directly to other stateful, multi-thread control-plane services of this kind. The part is also small enough to specify in full and build twice. That is what makes a comparison with matching tests possible.

The interface is small and follows the life cycle. It can create and release a context, set up a connection, handle a signalling message during the session, add a data radio bearer, begin, complete, or abort a reconfiguration, and look up a context's state under load. Each operation is defined for each state of Figure 3.1. An illegal operation returns a typed error and leaves the context unchanged. That closed set lets the same tests and the same workload drive both versions along the same paths.

RQ2 needs a fixed catalogue of bugs, not a loose list of hazards. One is built from this operation set: 14 types, in two groups of seven. Seven come from inside the region that safe Rust claims to govern. Seven were chosen because safe Rust is expected not to prevent them. Each expectation was written down before the programs were run. Section 5.3.4 reports what each one did.

The tail is the metric for two reasons. First, one action the user sees, such as a call setup or a handover, becomes many control-plane operations that depend on each other. The slowest one decides when the action is finished. A busy site drives thousands of such procedures per second, so a far-tail point is reached often enough to matter. Second, the shape of failure matters. Under an open-loop clock the sender keeps to its schedule and does not wait for replies. The achieved rate can then track the offered load until either the service, or work inside the timed sender, becomes the limit. Delay then rises sharply. That is why this study sweeps past the apparent saturation point, the load where the service stops keeping up, and checks what the timed path actually contains.

3.2 Research approach

The study runs experiments. It also builds the thing it measures: a matched pair of versions, plus the harness that drives them. Both are built from one frozen specification. Evidence comes from three separate methods. A load sweep, which raises the offered rate step by step, contributes to RQ1 and RQ3. Controlled fault injection, which plants known bugs on purpose, answers RQ2. An FFI migration prototype answers RQ4. A weakness in one method does not carry into the other claims. For example, the host machine can move the tail. That does not change what the fault-injection result says.

Each question asks how one outcome depends on one variable, the language. The language

has to be the only planned difference between the two programs under test. That rules out the most common way to compare languages. In that form, two programs written separately are measured, and the gap is taken to be the language. Such a gap also carries the design, the libraries, and the skill of the authors. Building both sides from one specification removes that confound. Where a question can be answered with numbers, it uses numbers. Where it cannot, the answer still follows a fixed plan, in words. That is how this study treats the kind of guarantee in RQ2. RQ4 asks about the design cost of the application programming interface (API), the set of calls the rest of the system is allowed to make.

The closest earlier work is the five vendor-hosted theses of Section 2.2. They appear in no citation database. They had to be found by searching each university's own archive by hand. The same searches found no Finnish thesis on Rust in a radio-access setting. They found no indexed paper mentioning Rust from the vendor hosting this work. The search returned no study that measures the tail of a Rust version and a C++ version of the same stateful, multi-thread control-plane part under too much load.

Table 3.1 maps each research question to the method, the artefact, the data, and the metric that answer it. RQ1 and RQ3 share H1. H1 was worded to cover a speed tie both in delay and in resource use. That wording turns out to be the hypothesis's main weakness, because those are not one quantity. RQ2 does not depend on the host machine. RQ4 depends on the host along one axis. Build time is a timing, so it is read as a ratio between two toolchains measured in turn on one machine.

Table 3.1: How each research question is answered. RQ1 and RQ3 carry two methods rather than one. Below saturation the sweep measures the round-trip a deployment would experience. The split of the core from the handoff measures the part the language can own, while later diagnostics test whether high-load recorder work contaminates the sweep.

RQ	Method	Artefact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak resident set size (RSS), CPU, throughput
RQ1, RQ3	Core called directly, then through the service	Matched microbenchmark pair	operation vs. cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, three C++ sanitisers, `rustc`, Loom/Shuttle, Miri	compile-time / runtime / undetected
RQ4	FFI migration prototype	C++ host + Rust C-ABI boundary	boundary size, `unsafe` scope, per-call cost

3.3 Methods per research question, in outline

RQ1 and RQ3 use an open-loop load sweep, from light load to well past saturation. Open-loop means the generator keeps to its send schedule instead of waiting for replies [38]. The four build variants take turns inside each repeat, so none of them always runs first. A variant here is one combination of language and memory allocator. A second timing splits the core operation from the cross-thread handoff around it. The end-to-end round trip bundles two costs of two kinds. Only one of them can differ between the languages. A third timing takes the contended queue and the reply handle on their own. Later full-grid diagnostics move or remove the histogram fill from the timed sender path. The histogram fill is extra bookkeeping that records how late a call was. Together these tests narrow the overload explanation. The comparison across setups does not isolate one cause.

RQ2 is answered by controlled fault injection [39]. Each bug type is a small program of its own, written in both languages. The C++ side is built and run under AddressSanitizer, ThreadSanitizer, and UndefinedBehaviorSanitizer. A sanitiser is a checking tool that watches the running program for one family of bugs. The safe-Rust side is compiled and, where it compiles, run under a bounded timeout. No static analyser is run, on purpose. The C++ columns then report the tooling this study ran. They do not report a ceiling on

what C++ tooling could find. RQ4 is answered by a working prototype in the direction a real migration takes. A C++ host calls an unchanged Rust core across a C ABI. This boundary is called the FFI seam here. It is the thin layer that converts between the C++ host and the Rust core. The per-call cost is measured on a real operation. It is not inferred from an empty crossing. Memory bugs that cross a language sit at that seam [40].

3.4 Fairness as a controlled variable

A language comparison is only as good as the things it holds fixed. The specification is frozen before either version exists. The thread model, the allocator and its arena count, the optimisation level, the hash function, the histogram method, and the load schedule match across the two sides. They are never tuned per side. Where the two languages could not be written alike, the gap is logged instead of quietly resolved. The two sides are shown to match by comparing logs operation by operation. For a given seed they also run the same operations in the same order. The four variants take turns on the same pinned cores inside each repeat. That keeps recent cache contents and recent branch guesses close, because the same work runs one after another. The test does not empty the cache between variants. It also does not first train the CPU on those branches. It holds the work and the cores fixed. It does not hold the CPU's recent memory and branch history fixed.

3.5 Statistical treatment

The statistical design was fixed in advance. The family of comparisons to be tested was listed before the campaign ran. A load picked after seeing the results cannot join that family. Each comparison is an exact paired permutation test over 20 interleaved repeats, with a bootstrap confidence interval [41], and Holm correction across the family [42]. In plain terms, the two languages are compared on the same repeats, the test asks how often a gap that large would appear by chance, and the Holm step stops a long list of tests from inventing a false hit. The floors the test can reach (0.0000019 uncorrected, 0.000027 corrected) are quoted so that a reported null can be told apart from a mathematical limit of the test. Throughput at the representative load is reported but not tested, because it is pinned to the offered rate. If the service keeps up, throughput equals the rate that was sent. Jitter, the small wobble from call to call, is outside the research questions. The maximum is too sensitive to one disturbed operation for pairing to rescue it. The pooled p99.99 far tail uses the operation as its unit, so it has no interval across repeats to test. No

latency percentile is tested after saturation. Past that point, the sender's schedule drifts, and the recorded percentile becomes a lower bound rather than an estimate of how long the service itself took.

3.6 Reproducibility and research artefacts

A third party can rebuild the results without access to Nokia's private code. The versions are driven entirely by the public frozen specification. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published with the thesis. Each plot of measured data is rebuilt by one script from the raw logs. So is each quoted number in the results. The script is `bench/refresh_thesis.sh`. That claim does not cover the hand-drawn diagrams, which explain a concept rather than report a result. Running the same scripts over another host's campaign rebuilds the artefacts by the same procedure. It does not reproduce the same values.

The logs that the script reads are part of the package. Each campaign has its own script and its own macro prefix, so a figure from one campaign cannot leak into a sentence about another. Each run's host setup is recorded and stored beside the campaign data. Nothing in the results chapters is typed from those files by hand.

4 Design and Implementation

Nothing is built until the specification is frozen. It fixes the public API, the UE state machine, the data layout and the steps on it, the locks and the shard size, and the thread model. Both versions follow it. The test then times the language, not a gap of design or of skill. The specification is itself an output of the study. It is the main control against the bias of the person who wrote both versions.

One thread model is used on both sides: shared-nothing, with one thread per shard. Shared-nothing means the workers do not share the UE records. A shard is one slice of the identifier space, owned by one thread. That layout is known to change tail delay by itself [43]. The layout is held fixed so the comparison stays fair. A production gNB-CU-CP schedules UE work differently. The identifier space is split into shards. Each shard is owned by one worker thread. That thread alone reads or changes it, so no lock guards context state. Callers send a request to the owning shard's bounded queue and wait on a one-shot reply handle. The queue blocks when it is full. That is on purpose. A rule that dropped requests would hide overload. This study needs overload to show up as delay. The only shared mutable state either version allows is one atomic counter. No control flow depends on its value. Figure 4.1 shows the data flow. There is one multi-producer, single-consumer (MPSC) queue per shard. Many senders can write to it. Only the shard's worker reads it. The worker drains it first in, first out (FIFO).

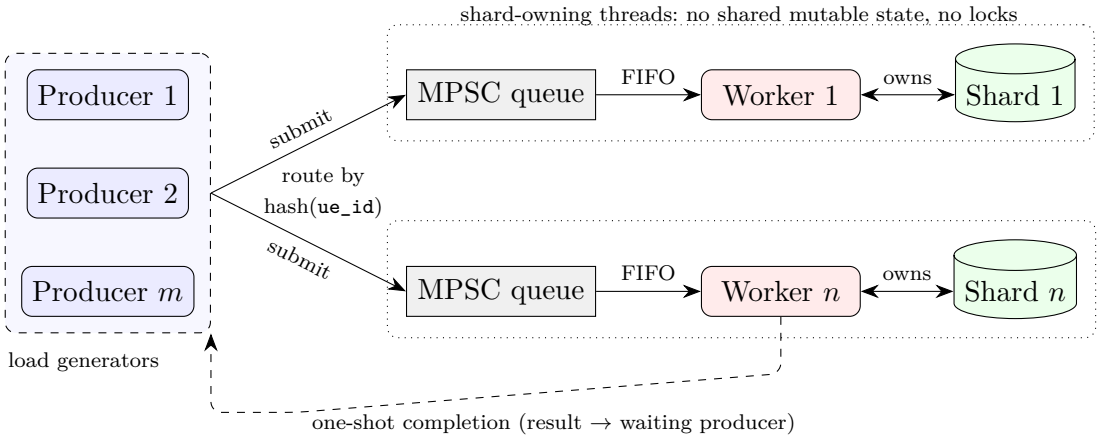


Figure 4.1: Shared-nothing, thread-per-shard architecture. Producers submit to a bounded MPSC queue per shard, and that shard's worker drains it.

Fairness is kept below the design as well as in it. Both builds link the same `jemalloc`, which is a replacement memory allocator. Both also keep a build that uses the system's

own allocator, as a control. Both compile at the same optimisation level. Both record delay through one high-dynamic-range (HDR) histogram, set up the same way [44]. An HDR histogram can record both very short and very long delays without losing the slow tail. Hashing is pinned on both sides. SplitMix64 assigns the shard [45]. FNV-1a, the Fowler–Noll–Vo hash [46], is used inside the map. C++'s default hash of an integer is often just the number itself. Rust's default hash is deliberately stronger. Left as they came, the two standard libraries would have been part of the measurement.

Where the languages cannot meet a need the same way, the gap is written into a forced-idiom log and then measured. Two of those gaps were candidates for the overload result of Section 5.3.2, the result read from the campaign clock. One is the bounded queue. Rust takes it from `sync_channel`. C++23 still has no such primitive, so C++ builds one from a mutex and two condition variables. The other is the reply handle: `std::promise` on the C++ side, against a fresh `mpsc` channel per request on the Rust side. Tests of those primitives, and a replacement reply handle, do not reproduce the two large campaign gaps. A third gap carries the core-operation result of Section 5.3.1. The per-shard container is each language's own standard hash map. A fourth is the request encoding. C++ has no payload-carrying enumeration in this idiom, so a request becomes a tagged struct that holds every field any operation might need. That struct is 160 bytes, against Rust's 96. The net result is a Rust core 20% longer in lines of code. Its branching complexity is 13% lower [47]. The Rust library contains no `unsafe` at all. `unsafe` is the Rust keyword that switches the compile-time checks off for a marked region.

Parity is tested. It is not assumed. Language-neutral test vectors run in both versions, and their logs are compared operation by operation. A comparison of the two logs cannot tell "both correct" from "both wrong in the same way". A third version, in Python, acts as an independent oracle. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A campaign of 9 000 generated operations found no mismatch. The C++ side runs under AddressSanitizer, ThreadSanitizer, and UndefinedBehaviorSanitizer. The Rust side runs under Miri, an interpreter that checks Rust's safety rules as the program runs. A clean sanitiser result only covers the code that actually ran.

5 Experimental Setup and Evaluation

This chapter starts with the test machines and the work they run. The results then follow by load, and then by the rest of the questions.

5.1 Testbed setup

Some numbers in this chapter come from the tools and the code. Other numbers depend on the machine they ran on. The reported campaign comes from one bare-metal host, a machine with no virtual machine in the way, a single-socket AMD EPYC 7443. Each core runs one thread. Extra threads on a core are off. The host runs Ubuntu 24.04.4 LTS on kernel 6.17.0-23-generic. Linux can raise and lower each core's clock as load and heat change. The clock is held fixed. The governor is performance, and boost is off. The test may use only cores 2–17. There is no hypervisor. The program runs on the host itself, not inside a virtual machine. On those cores the kernel turns off its timer tick (`nohz_full`). If left on, those ticks would show up in the delay numbers. The study does not use `isolcpus`. That flag also stops the kernel from sharing work across the allowed cores. Each test thread then lands on one core.

The build tools are not the ones installed on the host. Both sides are built and run inside an image that carries Rocky Linux 8.10 (Green Obsidian) with `g++ 15.2.1` and `rustc 1.97.1`. The image pins the `gcc-toolset` major version, not the point release. The earlier sweep on a virtual machine then built with `g++ 15.1.1`, one GNU Compiler Collection (GCC) 15 release earlier. An image still uses the kernel of the host it sits on. The two sweeps then differ in the hardware, in the kernel (6.17.0-23-generic against 7.0.2-1.el8.elrepo.x86_64), and in that compiler release.

Until late in the work the sweep ran on a Kernel-based Virtual Machine (KVM) on an Intel(R) Xeon(R) CPU E5-2680 v4, with none of the core controls above. The capture records the hypervisor as kvm, the guest userland as Rocky Linux 8.10 (Green Obsidian), and the guest kernel as 7.0.2-1.el8.elrepo.x86_64. On that host the 95% band on one variant's p99 spans 1480 μs. The four variants' p99 values differ by 293 μs. The noise on one variant was larger than the gap between all four. That sweep is kept. It uses the

same design as this one. The two can then be set next to each other to ask which findings belong to the code. A third sweep, on a second bare-metal AMD EPYC 7443P, uses a new shard count, load grid, run count, and tools. It tests whether the findings hold when those things change together, not any one of them on its own.

5.2 Workload and measurements

The work is made for the test. The steps it takes come from the 3GPP rules [1, 4]. A driver that always sends the same steps walks each UE through the connection life cycle those rules define. The *signal* workload first connects a set of UEs, then sends the same signal call again and again. That picks out the common path. The *mixed* workload walks each UE through context creation, setup, signal, bearer setup, reconfiguration or handover, and release. A fixed random order makes both sides see the same counts of each call kind for a given seed. That is also a check that the two sides did the same work. Figure 5.1 shows the two loads and the steps the mixed one walks.

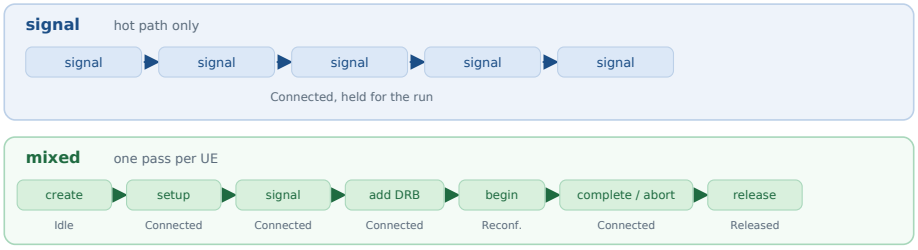


Figure 5.1: The two workloads, and the lifecycle the mixed one walks. Signal repeats the common path, and mixed walks each UE through the procedure sequence drawn below.

The steps match the 3GPP rules. Those rules do not set the mix of call types. A real cell's handover rate depends on the site and the time of day, and no public figure fixes it. A mix that does not match a real cell moves both sides by the same amount. It limits how far the numbers travel to a real network. It does not make the test unfair. The sender keeps to its plan even if the service is late. That is the open-loop rule. The histogram is corrected so a stall does not hide missing replies. That correction is for coordinated-omission bias. If the recorder only times the calls that finished, it misses the ones that were stuck [38, 44]. Each load is run 20 times. The four language and heap variants take turns inside each run. Each point is a median with a bootstrap 95% band [41, 48]. The band is the range in which the median would be expected to fall if the experiment were repeated.

The load used for the resource plots is 56 kilo-operations per second (kops/s). It is a chosen value, not one derived from the sweep. The first rule this study used was the peak

load whose p99 stayed under 5 ms. On the virtual machine that rule landed on the noise of the host. On a quiet host it climbs until the service can no longer keep up. It is kept only as a check. The lowest finished rate among the four variants is 392 kops/s, reached when the sent load is 392 kops/s. That finished rate is the knee: the point where raising the offered load no longer raises the rate the service completes.

5.3 Results

The safe-code results, and most of the migration results, do not depend on which host was used. The speed and memory results do. Each figure below was timed on the host of Section 5.1 unless the caption says so.

5.3.1 Below saturation (RQ1, RQ3)

Saturation here means the service can no longer finish work as fast as it is offered. At the representative load of 56 kops/s, every variant still keeps up with the sent load. The four variants' medians lie within 1.6 μs of each other, a spread of 2.2%. A paired test still finds a gap of 1.3 μs in Rust's favour, under both heaps. Figure 5.2 shows the sweep behind that number. The higher tails can now be read as well. The widest band on one variant's own p99 spans 0.8 μs. The four variants' p99 values differ by 1.6 μs. So the noise on one variant is smaller than the gap between them. On the virtual machine the same two sizes were 1480 μs against 293 μs, the other way around. Both p99 rows and both p99.9 rows clear the Holm correction, 4 rows in all, in the same direction as the median. Table 5.1 reports p99.9 and the maximum at the chosen load. The maximum is kept out of the tested family, as the pooled p99.99 is.

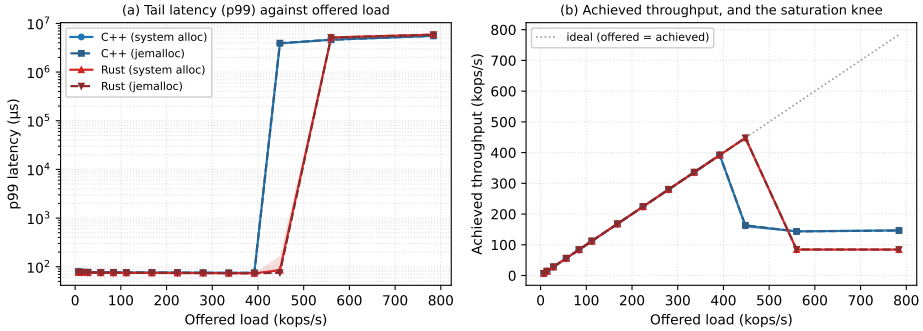


Figure 5.2: The load sweep, both panels against offered load. Lines are medians over 20 repeats, and the bands are bootstrap 95% confidence intervals.

The far tail can be read here too. The runs' histograms are pooled, which gives 3 360 051

Table 5.1: Per-operation latency, in microseconds, at 56 kops/s. The median over 20 repeats sits above its bootstrap 95% interval. †The p99.99 column is pooled across repeats, so it has no interval. The maximum has a repeat-wise interval but stays outside the tested family because one disturbed operation can determine it.

			(µs)		
Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.7	76.4	81.2	142.0	243.1
	[73.5, 73.7]	[76.2, 77.0]	[80.8, 81.6]		[239.2, 275.8]
C++ (jemalloc)	73.5	76.6	80.2	176.4	246.3
	[73.3, 73.6]	[76.4, 76.9]	[79.5, 82.0]		[228.8, 259.5]
Rust (system alloc)	72.4	75.0	77.3	92.1	145.2
	[72.3, 72.5]	[74.7, 75.2]	[77.0, 78.7]		[115.9, 178.2]
Rust (jemalloc)	72.1	75.6	77.8	89.5	121.4
	[72.1, 72.3]	[75.6, 75.7]	[77.7, 77.9]		[101.8, 179.6]

to 3 360 084 calls per variant. Pooling means the calls from every repeat are put into one histogram, so the very slow end has enough samples to see. The two Rust variants sit below both C++ variants. The widest pair differs by 87 μs. The slowest Rust variant still sits 50 μs below the fastest C++ one. Below here means faster: a smaller delay. A second bare-metal host, at another shard count, agrees. It runs 147–228 μs for the C++ variants against 88–115 μs for the Rust ones, and the ranges do not meet. A second limit does not come from the host. No host would remove it. An end-to-end round trip spends only 0.250% of its time on the work the language does. The rest is passing the request to another thread, and that time is charged to the operating system. When the work inside the language is this small, the time spent crossing from thread to thread dominates [49]. Figure 5.3 draws that split. This sweep cannot bound a language effect either way. Figure 5.4 sets that limit beside what the sweep can still show. Timed on its own, the core call is 1.6× faster in Rust. The test does not trace the cause. The stock hash map is one candidate. The compiler's code generation may also own part of the gap.

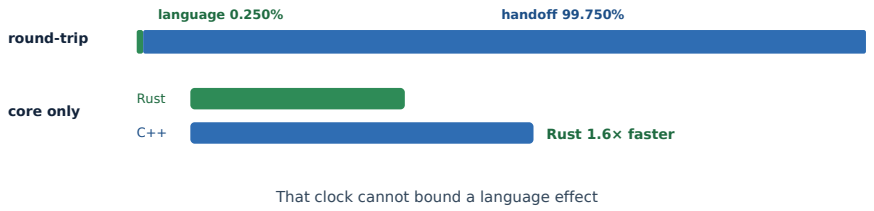


Figure 5.3: Where an end-to-end round-trip's clock goes. The language share is 0.250%. A clock that spends the rest on handoff cannot bound a language effect.

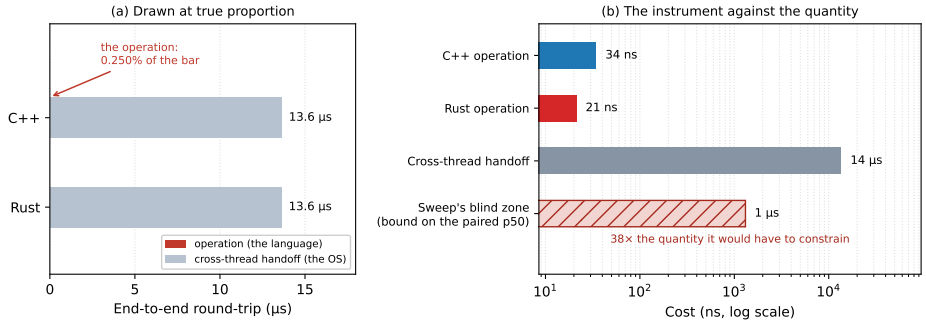


Figure 5.4: Where an end-to-end round-trip's cost goes. **(a)** The round-trip at true proportion. **(b)** Both operation measurements sit inside the sweep's blind zone by a factor of 38.

5.3.2 Past saturation (RQ1)

The campaign clock is the timer used in the main experiment. It includes the service and the recorder that writes the histogram. Under that clock, the four variants appear to stop keeping up at loads that do not match. The Rust versions hold to 448 kops/s. The C++ versions hold to 392. Those two loads are the knees reported by that instrument. So at 448 kops/s, Rust still serves 448 kops/s where C++ has fallen to 162. That load is shown and not tested, because it was found after the data was in hand. A comparison chosen after seeing the data cannot join the pre-planned test family. Figure 5.5 maps the representative load, both knees, and the overload point.

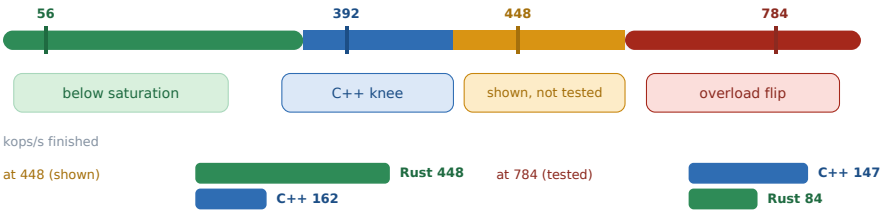


Figure 5.5: Four operating points on the offered-load axis, spaced for reading, not to scale. Ticks are 56, 392, 448 and 784 kops/s. The comparison at 448 belongs to the campaign clock and is absent on the later deferred-fill setup. Bars are campaign-clock throughput there and at 784 kops/s.

Deeper into overload, at 784 kops/s sent, the order flips. C++ still finishes 147 kops/s where Rust finishes 84. That is a factor of 1.7, in 20/20 runs under both heaps. The Holm-corrected p is as small as this test family can go, which means the gap is larger than the test can fail to see. The slower variant uses 1.8 times the CPU for each call it finishes. That quantity belongs to the whole campaign process, including the recorder. It does not identify work inside the service. The bounded queue and the reply handle do not

reproduce the order when they are timed alone.

Two later sweeps test the recorder explanation on a separate captured setup. Each repeats the fourteen-load grid over 1120 runs, holding the pinned cores, the allocator pair, and the repeat count. The processor is AMD EPYC 7443P, against AMD EPYC 7443 for the campaign. The later setup has none for `nohz_full`, against `2-17` in the campaign. When a sender falls behind, the harness writes up to 10 000 extra samples for each finished call. That extra writing is the fill. Clock A drops that fill and starts the timer at the actual send. Its percentiles are coordinated-omission biased on purpose, so its lower Rust p99 is a diagnostic hint. It is not an RQ1 tail result. Clock B keeps the intended send time and the cap, but writes the fill after the timed window.

Neither diagnostic repeats the large collapse. With the fill delayed, both languages keep up through 448 kops/s. At the top load the system-allocator medians are 449 kops/s for C++ and 455 for Rust. The full 20 paired repeats cannot tell them apart, at $p = 0.075$. Under `jemalloc`, Rust serves 467 kops/s against 445 for C++. The full 20-pair test resolves that smaller Rust lead at $p < 0.001$. These later p-values were computed after seeing the data. They are not adjusted, and they do not join the pre-planned Holm family. Percentiles on Clock B are the sender falling behind its schedule. They are not the service's own time, so throughput carries this diagnostic.

Clock B also lacks the campaign's uneven knees. At 448 kops/s the campaign has Rust serving 448 kops/s where C++ has fallen to 162, a factor of 2.8. With the fill delayed, C++ serves 447 kops/s and Rust serves 448. One gap favoured each language in the campaign. Both disappear on the later setup. That pattern fits recorder work as the cause. The host and the kernel setting also changed, so the study cannot claim that the fill timing was the only cause. The campaign stays the pre-planned comparison. The later sweeps change how its high-load gaps are read. They do not give service-latency percentiles after saturation, and they do not establish a general ranking of top rates. Figure 5.6 compares both clocks at both loads and keeps the two heaps apart.

5.3.3 Resource use (RQ3)

The four variants differ in RAM and CPU even where their median delays are close. Peak resident set size (RSS) is the most memory the process held at once. On the reported host it contains two effects of similar size. The Rust build uses 1.4–2.5 MiB more peak RAM than the C++ build, under either heap. Linking `jemalloc` costs 1.5–2.4 MiB in either language. Figure 5.7 shows both effects. The language effect keeps its direction and its scale in megabytes on all three hosts, but not one exact range. It spans 1.8–3.5 MiB

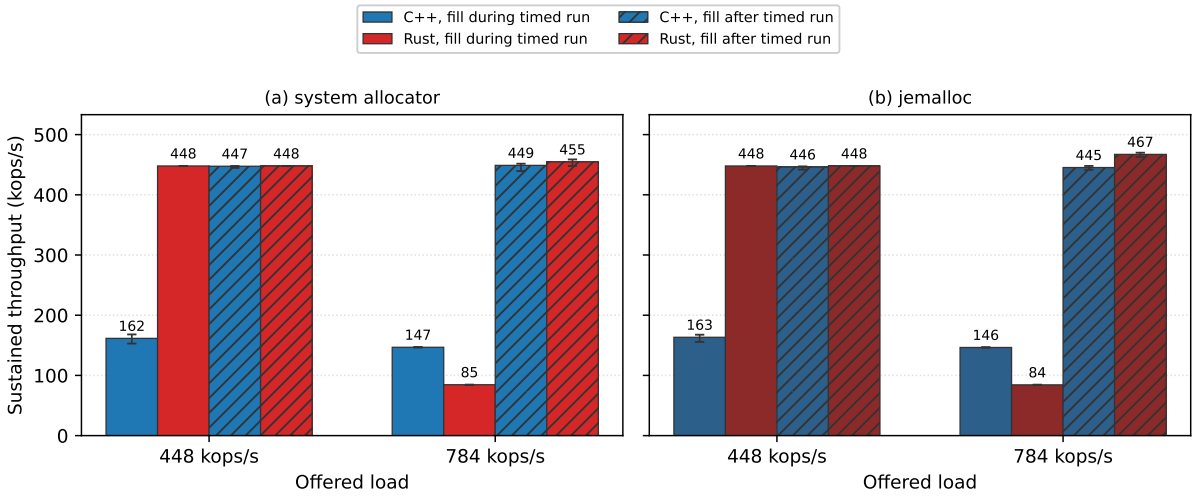


Figure 5.6: Throughput at the two loads where the campaign shows opposite language gaps. Bars are medians over 20 interleaved repeats with bootstrap 95% intervals. Solid bars are the AMD EPYC 7443 campaign with `nohz_full` on `2-17`. Hatched bars are the later AMD EPYC 7443P diagnostic with none for `nohz_full`, where the same fill runs after the timed window. Panels keep the two heaps separate. The later setup does not reproduce either large campaign gap.

on the guest and 0.7–1.8 MiB on the second bare-metal host. The allocator effect's size does not travel either. On the virtual machine it was 13 times the language effect, 32.5–34.3 MiB. Their ratio is therefore a property of the host. Allocator accounting on the second bare-metal host places the language gap outside the heap. Rust has +2.07 MiB more arena metadata, but −0.16 MiB less live allocation and −1.92 MiB less allocator-resident memory, while the process peak remains +1.74 MiB higher. The extra RAM is a fixed cost of the process. It is not more live application data. CPU is a separate result. At 56 kops/s Rust uses at most 5.1 percentage points less. At 784 kops/s the slower campaign-clock variant instead uses 1.8 times the CPU per finished call. The deferred-fill sweep does not reproduce that large flip. That supports a recorder-work explanation, without separating it from the changed setup. Counters on the second bare-metal host cover only the isolated core call. Rust retires 11% fewer instructions and finishes in 15% fewer cycles, while missing the cache 29% more often per thousand instructions. Those counters explain the isolated core result. They do not explain the overload floor.

5.3.4 Safety via fault injection (RQ2)

Safe code is checked by planting 14 bugs. Seven come from inside the zone Rust claims to govern. Seven were chosen because safe Rust is meant *not* to stop them. Each of the first seven is a compile error in Rust. In C++ each is valid code. The bug shows up only if a test drives it, and no single sanitiser hits all seven. Use-after-move is the case where

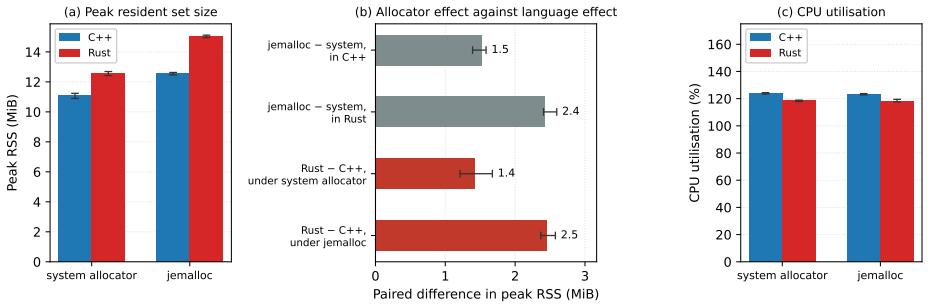


Figure 5.7: Resource use at 56 kops/s, grouped by allocator. Bars are medians over 20 repeats with bootstrap 95% intervals, and CPU in (c) is summed across threads.

no sanitiser flags anything. A moved-from object is still a valid object. Its value is unset. There is nothing illegal to detect. The name that used to point at the value is simply gone. All seven boundary cases compile in safe Rust, and then fail when they run. On two of them the C++ tools are the ones that report the bug. LeakSanitizer flags the reference cycle that Rust leaks in silence, because leaks are safe in Rust by design [50, 51]. UndefinedBehaviorSanitizer flags the overflow that Rust defines as wrapping around. ThreadSanitizer catches the lock-order inversion in 3 of 12 runs, a rate that itself jumps around. It logs lock orders only from locks that return. If the first inversion waits and never returns, the cycle is never closed, so the tool never flags it.

The seam double-free is the in-guarantee double-free written again through `unsafe`. It is not a new bug type. Its C++ half is the same, so only the Rust column moves. Two Rust model checkers do flag both of the original boundary bugs. Loom explores thread orders in a way that does not miss a scheduled interleaving. Shuttle picks orders Loom cannot reach [52]. Each bug had to be rewritten in the checker's own parts first, and shrunk to model size. Miri flags the seam double-free, but it reads Rust and cannot run the C++ host the bug sits behind. LeakSanitizer and UndefinedBehaviorSanitizer flag without a rewrite. The two model checkers flag the concurrency boundary cases only after the bug is recast into their parts. Figure 5.8 draws the whole list as a matrix of outcomes. Resource Acquisition Is Initialization (RAII) ties cleanup to how long an object lives. It removes the plain missed unlock, because leaving a scope destroys the guard and releases its lock. It does not remove the bug type. Both forgotten-guard programs hang, and neither side's tools said so.

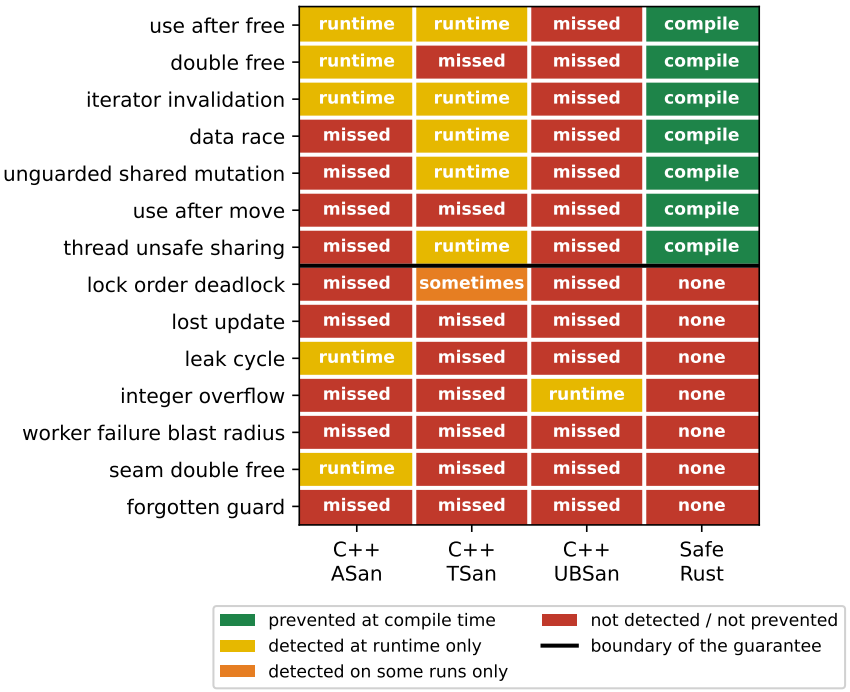


Figure 5.8: The fault-injection outcomes as a matrix, generated from the taxonomy log.

5.3.5 Migration (RQ4)

The FFI seam is the C-ABI layer between the C++ host and the Rust core. It is small on each measured axis: 11 C entry points in 231 lines of Rust. All raw-pointer work is kept in 9 functions. The core is left free of `unsafe`. No extra cost is seen against the same signal call made directly. That bounds any per-call tax at 10.91% of that call. Figure 5.9 shows the structure that bound belongs to. Three adoption costs beyond the seam were measured as well, and two of them go against the usual worry about taking Rust. 54% of the Rust clean build is third-party code. The project's own source builds in 0.8 times the CPU time that `g++` needs. The 6.8× size gap is a choice of how libraries are linked. The order flips when that choice is held fixed. The compared part has no third-party crates and no `unsafe` sites (0 crates, 0 `unsafe` sites).

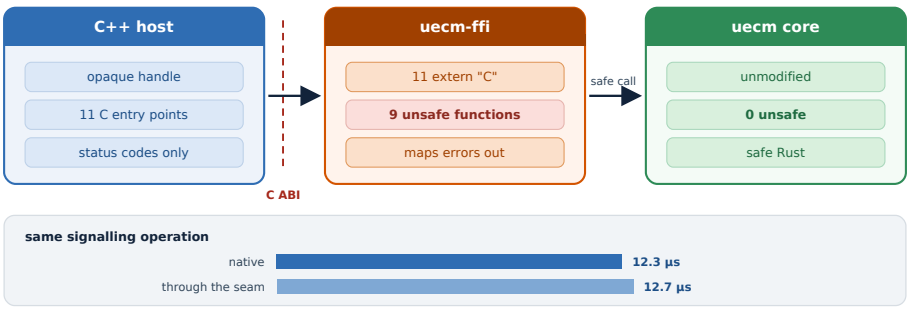


Figure 5.9: The C++-to-Rust FFI migration seam. The same signalling operation costs 12.3 µs directly and 12.7 µs through the seam, a paired difference bounded at 10.91% of that call. A do-nothing crossing costs 2.54 ns and bounds only itself.

6 Discussion

6.1 Summary of findings

At 56 kops/s the variants agree to within 1.6 μs at the median. The pooled far tail lands on Rust's side, which means the slowest calls are faster there. The end-to-end round trip is only 0.250% language work, so it cannot say how large a language effect is. The same core, timed on its own, is 1.6× faster in Rust. At higher loads the campaign shows one gap for each language. Rust leads at 448 kops/s. C++ leads at 784 kops/s. A later captured setup, with the histogram fill delayed, reproduces neither large gap. That fits recorder work as the cause. It does not separate that cause from the change of machine. Neither overload clock supplies percentiles of the service's own latency, so RQ1 has no language verdict after saturation. The RQ2 outcome is complete for the fourteen bug types chosen in advance, and it does not depend on the host. All seven in-guarantee types are compile errors in safe Rust. All seven boundary types compile and then fail, in Rust and in C++. RQ3 finds a higher Rust peak RSS under both heaps: 1.4–2.5 MiB on the campaign host, 1.8–3.5 on the guest, and 0.7–1.8 on the second bare-metal host. Below saturation, Rust uses slightly less CPU. Corrected high-load throughput gives no general language ranking. RQ4 finds a small seam. Its per-call tax is bounded at 10.91% of that call. API design and learning remain adoption costs that earlier papers describe. This study does not measure their size.

6.2 Discussion on the research questions

RQ1 has two answers, split by how hard the service is working. At 56 kops/s the campaign resolves differences at the median, at p99, and at p99.9, and the pooled p99.99 lies on Rust's side. The end-to-end clock still cannot say how large a language effect is, because the core call is only 0.250% of what it times. After saturation, neither available percentile clock estimates the service's own latency. The campaign includes fill work on the sender side. Clock B measures the sender falling further behind its schedule. No ranking of the overload tail follows.

The campaign throughput shows Rust leading at one step and C++ leading later. The later captured setup reproduces neither large gap when the fill is removed or delayed.

That is evidence for recorder work. The changed processor and the changed kernel setting stop the study from naming fill timing as the only cause. Clock A’s lower Rust p99 is deliberately biased by coordinated omission. It is not an RQ1 tail result. Clock B’s throughput leaves the system-allocator pair unresolved, and it leaves a smaller Rust lead under `jemalloc`. The closest earlier work reports a similar worsening past a threshold rate and attributes it to stackless `async/await` [32]. That cause cannot apply here, because neither side uses an asynchronous runtime. Changing Rust’s reply handle does not move the floor either. The campaign ratio is therefore a result about its harness. It does not rank the top rates of the two services. The prediction written down in advance had named bounds checks and overflow checks as the cause [53, 54]. That part was wrong. At normal load the median result still fits evidence that modern out-of-order CPUs absorb most of Rust’s safety checks [12, 35].

Some safety bugs are compile errors, so they never become a program that can run. The seven types Rust does not stop are part of the answer. They were chosen before the runs. A compiler that will not build a program keeps the bug out of every run. A sanitiser keeps it out only of the runs a sanitiser test happens to take. What changes between those two is how much of the running program is covered. The list of bug types stays the same width. Figure 6.1 names the mechanism on each side of that line. Rust reasons about aliasing, which is whether two names refer to the same memory. It does not reason about the order of locks. A `Mutex` guard whose lifetime the borrow checker tracks well says nothing about whether two threads take their locks in a matching order. Static deadlock checks for Rust are an open research problem [55]. The lost update misses ThreadSanitizer for the same reason it misses the borrow checker. Where each access is properly synchronised, there is no unsynchronised access to find. In a field study of 105 real concurrency bugs, about a third of the cases that were not deadlocks broke the order the writers meant, not their locking [56]. Adopting Rust removes data races. It leaves in place the concurrency bugs that fail quietly. A migration plan that reads “memory-safe” as “concurrency-safe” would spend its review effort on the wrong bugs. In the corpus of Xu et al., every published Rust memory-safety entry in Common Vulnerabilities and Exposures (CVE) involves `unsafe`, apart from one compiler bug [57]. That is why keeping the core free of `unsafe` matters. It is also why `unsafe` spreading through library crates is a review concern [58].

One case reaches past the small fault programs. A gap in the specification let a call be sent outside the window when the service was running. It passed each test, each parity vector, and all three sanitisers, because nothing ever called that path. What turned undefined behaviour into a catchable failure on the Rust side was an ordinary bounds check. It was not a compile-time proof. The useful thing to plan for is checks that are on by default.

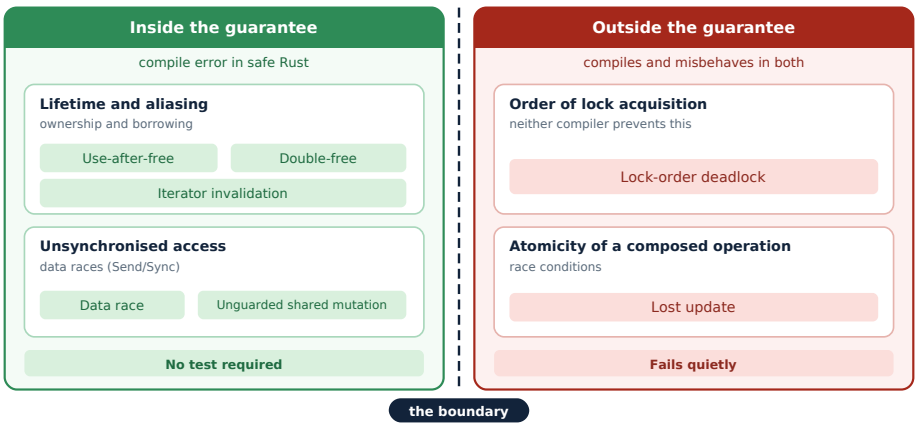


Figure 6.1: Where Rust's compile-time guarantee stops, with classes grouped by the mechanism that rejects them.

A claim that the compiler has ruled the bug out is stronger than this case supports. The root cause belongs to the method. A ban written down with no mechanism behind it leaves each version to fail in its own way.

RQ3 splits by resource, and the splits do not point the same way. The RAM cost holds under both heaps and on a second host with a new compiler. Peak RSS is a maximum over the whole run, so the jitter that ruins a p99 does not touch it. Set beside the heap's own effect, the two are of the same order on both bare-metal hosts. On the guest the heap's effect was an order of magnitude larger. The language effect travels from host to host. The ratio of the two effects does not. The quiet host also places the cost outside the heap, by the heap's own counts. One close study reports the other direction, with Rust lowest and the gap growing as concurrency rises [29]. Those two findings agree once the variants are read as two concurrency runtimes. Their C++ runs on Boost.Fibers, which gets a stack per fiber. Their Rust runs on Tokio, where a task is a small future on the heap. What they timed is the per-task cost of two concurrency runtimes. Two further studies agree with the direction found here [30, 59]. Below saturation, Rust uses slightly less CPU. The larger flip under the campaign clock is not reproduced with deferred fill, and it cannot be assigned to Rust's safety checks.

RQ4 finds a small seam on the machine side. A C++ host can call an unchanged Rust core across a thin C-ABI seam. The cost on a real signal call is bounded at 10.91% of that call. Earlier work names design and learning as human costs [60, 61]. This study does not measure their size. Keeping `unsafe` at the seam bounds what has to be reviewed. It does not bound what can be corrupted, because the host and the part share one address space [62, 63]. The build time and the binary size point the same way as the seam. A vendor worried about either should audit its crate budget before blaming the compiler.

The one axis where C++ leads outright is the edit-and-compile loop, for changes that stay inside one source file. The 6.8× size figure compares two linkage policies. The order flips when the policy is held fixed.

6.3 Revisiting the hypotheses

Section 1.4 set three hypotheses before any run. Saying how they came out is what makes this a test.

H1 It is not accepted as written. No margin for “close enough” was defined, so a small but statistically detectable median difference cannot establish that the two languages are equally fast. At 56 kops/s the paired test resolves 1.3 μs, and it resolves differences at the higher percentiles. The pooled far tail favours Rust. The isolated core favours Rust. After saturation the available clocks do not provide a valid ranking of the service’s own latency. The hypothesis folded the round trip, the core code, the recorder, and the overload behaviour into one quantity. The method cannot support that single quantity.

H2 For the seven injected types inside the guarantee, it holds with no exception. It fails if it is read more loosely than it was written. It is accepted for memory safety and for freedom from data races. It is rejected for concurrency correctness in general. It is a claim about *safe* Rust only. The seam case shows the same bug type coming back inside `unsafe`.

H3 Any per-call tax is bounded at 10.91% of that signalling call. The support is the upper bound on the paired difference. A difference that fails to reach significance would not, on its own, show that the tax is small. The accompanying claim, that the real cost is design and training, is supported by the shape of the result. It is not a statistical test.

6.4 Engineering guidelines for incremental adoption

The rules below are the practical form of the findings. Each one carries the limit of the finding behind it. Choose a first part for how thin its API is. Size is the weaker guide. Rewriting a rich interface is the schedule risk. The language boundary itself is small. Keep the C ABI in its own crate, and keep the core free of `unsafe`. That keeps `unsafe` out of the chains of crates it otherwise spreads through [58]. Leave the concurrency review

budget where it is. Prefer a shared-nothing structure in either language. The strongest concurrency result here belongs to the design. It belongs to neither compiler. Plan the learning curve as a training cost, and track it. Time the inner part as well as the round trip. Sweep past the point where the service stops keeping up. Audit every operation on the sender's timed path, including the histogram correction. Run recorder work after the timed window. Compare a clock change on the same captured setup before assigning a high-load gap to the service.

6.5 Where this study sits, and what it cannot support

The studies closest to this one are the five vendor-hosted theses of Section 2.2. Their verdicts clash with each other in every direction. The one with the opposite sign on peak RAM fits this study once its variants are read as two concurrency runtimes. Four threats still matter. The first is that one author wrote both versions. The same failure mode flipped a printed verdict in this field: the pair looked comparable after a code review, and then Rust was 85–91% worse after more speed work [33]. The parity suite bounds what the code does. It does not bound speed. The second threat is that the end-to-end clock times the design faithfully and the language hardly at all. Third, the later probes do not share the campaign's processor or its `nohz_full` setting. Their failure to reproduce the two large gaps supports a recorder explanation. It does not isolate fill timing as the only thing that changed. No peer-reviewed study reports that same instrument effect. Fourth, neither high-load clock gives percentiles of the service's own latency. The campaign includes recorder work. Clock A has coordinated-omission bias. Clock B reports schedule drift. The thesis therefore leaves the service's latency under overload unresolved.

6.6 Ethical and professional reflection

The study uses no personal data, no subscriber data, and no operator secrets. All traffic is made for the test. The service is based on the shape of live code. It is not copied from it, so the artefacts can be shared in public. The first duty is to avoid claiming too much. A vendor could read the RQ2 result as permission to describe a moved part as "safe". The boundary cases are why that reading would be wrong. A deadlocked control plane is an outage. A lost counter update that stays silent can mislead a mobility decision. Nothing shown here stops either of those. The second duty is to say how far the data reaches.

Tail-delay numbers from any single host are a thinner basis for a production roll-out than they look. That is why the earlier guest sweep is kept.

7 Conclusion and Future Work

The thesis set modern C++ against Rust. The object is a UE Context Management service. It stores the state of each device, as a gNB-CU-CP does. It looked at four things: how late the slow tail is, which bugs a compile step can stop, what the safety checks cost, and what it costs to add Rust a piece at a time. Both sides were built from one frozen specification. They were driven along the same code paths. That is what lets a language effect be separated from a gap of design or of skill.

The safety mechanisms act at different stages. Safe Rust rejects all seven injected memory-safety and data-race types before they become programs that can run. The same seven compile in C++, where a sanitiser can report a bug only if a test executes the relevant path. All seven boundary types compile and fail in both languages. On two of them the C++ tools flag a bug that Rust's tools do not. Rust therefore removes data races and aliasing errors. It does not remove race conditions, lock-order errors, leaks, wrapping arithmetic, or the blast radius of a failed worker. It removes nothing inside an `unsafe` block. LeakSanitizer and UndefinedBehaviorSanitizer flag two boundary bugs without a rewrite. The two model checkers flag the concurrency cases only after each bug is recast into their parts.

On migration, the measured seam on the machine side is small. An unchanged Rust core can be driven from C++ across an FFI seam, the C-ABI layer where the two languages meet. It has 11 entry points in 231 lines, and its 9 raw-pointer functions contain all of the `unsafe`. The core has 0 `unsafe` sites. The cost on a real signal call is bounded at 10.91% of that call. Earlier work names API design and the learning curve as adoption costs. This study does not measure their size. Most of the extra cost of a Rust build is compiling crates, not the compiler's own work on this project. The size gap is a linkage policy. The order depends on which matched policy is chosen. Where C++ leads is the edit-and-compile loop, and only for edits that stay inside one source file.

On speed, the data first splits by what is being timed. The end-to-end clock starts before a sender submits a request and stops after the worker replies. It includes the queue, the wake-up, and the core call. Only 0.250% of that round trip is the core work the languages perform, so this clock cannot bound a compiler effect. A second measurement removes the queue and the cross-thread handoff and times the core call alone. In that measurement the core is 1.6× faster in Rust. At the chosen load of 56 kops/s the paired test finds 1.3 μs in Rust's favour. At p99.99 the slowest Rust variant sits 50 μs below the fastest C++ one,

which means it is that much faster. The campaign then shows a Rust lead at the knee and a 1.7× C++ lead deeper into overload. A later captured setup with deferred fill reproduces neither large gap. That comparison fits recorder work. The change of setup stops a claim that fill timing was the only cause. The gaps do not rank the two services, and the service's latency under overload remains unresolved. On RAM, the Rust build carries a few MiB more of fixed footprint under both heaps. It does so in the same direction, and on the same megabyte scale, on all three hosts that were timed. The heap choice costs about as much on both bare-metal hosts, and an order of magnitude more on the guest this study used first.

Taken together, these support a narrow claim. It applies to stateful, concurrent, partitionable control-plane services of this kind. Partitionable means the work can be split so that each thread owns its own slice. At the loads where the service still meets the rate it was sent, the choice is which bug types a team wants to make impossible to write. The other half of the choice is whether the team will pay the design and training cost of getting there. Past that load, the campaign includes substantial recorder work and adds no second language ranking. What the thesis adds is the frozen specification, the pair that passed the same tests, and a timing harness that reaches past saturation. A split of the round trip says how much of the timed work the language can own. A bug list maps where the guarantee stops.

Future work answers the gaps one at a time. The first job is a same-host control that repeats in-run fill and deferred fill on one captured setup. It would separate recorder timing from the differences in processor and kernel settings. The next job needs no new host. It is to vary the transparent hugepage policy on one host, with everything else held fixed. A hugepage is a larger block of memory than the usual page, and the policy decides when the kernel uses one. All three hosts agree that the Rust build carries the larger footprint. They disagree about how that sits against the cost of the heap. The guest put the heap cost an order of magnitude higher, and neither bare-metal host does. The hugepage policy is the candidate this study can name but not test, because it was not the same on each host and only the system-allocator variant receives it. Beyond that, replaying anonymised production traces would check the mix of calls. The harness can also take further control-plane parts, or the data-heavy layer-2 and physical-layer (PHY) path. A part with truly shared mutable state would exercise the two sides in a way a shared-nothing design does not. Testing Rust async runtimes against the thread-per-shard model would separate the runtime from the language in the tail result [32]. Timing what it costs to split the two heaps inside one process would price what it is worth to keep a core free of `unsafe` [62, 63]. A longer case study would track bug counts and review effort while

Rust is added a piece at a time. That is what would settle the human cost this study can only name.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5 and Cursor Grok 4.6 are the generative AI systems used in preparing this thesis. Both ran through the Cursor development environment from June 2026 through the writing of this thesis. University of Helsinki policy forbids naming a language model as an author. Neither is cited as a source, which is the author's own choice.

What the assistance covered. The models drafted and revised the running prose, and proposed the order of the arguments. They wrote and refactored both versions of the UE Context Management service. They also wrote the load generator, the fault-injection catalogue, and the FFI prototype. The analysis behind the bootstrap intervals and the paired permutation tests is their work too, as are the figure, build and LaTeX sources. The abstract is this programme's maturity test. University policy never permits a language model in one. The abstract was written by the author unaided.

What the models did not settle. The research problem, the four research questions, the three hypotheses, and the study's main control are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only planned gap between the two versions. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names. The quantities the thesis quotes reach the page as macros rebuilt from the raw logs. The readings are the author's too.

How the output was checked. All model output, prose and code, was read by the author before it entered the thesis. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. An audit then graded how far each claim could be traced. The first grade is the page or clause itself. The second is a publisher-deposited record that carries the claim. The third is confirmed metadata behind a paywall. An entry whose supporting passage was never opened is listed, not absorbed. The audit found four passages that overstated or misplaced their source. One was a quotation absent from the work it was attributed to. All four were corrected. Code was checked against a conformance suite whose transcripts are diffed line by line. It was also checked against a fault catalogue that fixes each expected outcome before the defect is injected.

Responsibility. The author has reviewed the whole of this thesis and is responsible for all of it. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index

reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] A. Meneely, A. Green, T. Jaafari, M. Fluet, and B. Keller. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". In: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008.

[27] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[28] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[29] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[30] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[31] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[32] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[35] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[36] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[37] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are

affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[38] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[42] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[43] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[44] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[45] G. L. Steele Jr., D. Lea, and C. H. Flood. "Fast Splittable Pseudorandom Number Generators". In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*. 2014, pp. 453–472. DOI: 10.1145/2660193.2660195. URL: https://doi.org/10.1145/2660193.2660195.

[46] L. C. Noll, K.-P. Vo, D. E. Eastlake 3rd, and T. Hansen. *The FNV Non-Cryptographic Hash Algorithm*. Request for Comments (RFC) 9923. Informational. Specifies the FNV-1a hash used inside each shard. Internet Engineering Task Force (IETF), Feb. 2026. DOI: 10.17487/RFC9923. URL: https://www.rfc-editor.org/rfc/rfc9923.

[47] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[48] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[49] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[50] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[51] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[52] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[53] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[54] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[55] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[56] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[57] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[58] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[59] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[60] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[61] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[62] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[63] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.